

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA Nº 09-2

NOMBRE COMPLETO: VELARDE VALENCIA JOSUE

Nº de Cuenta: 320342330

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 07

SEMESTRE 2027-1

FECHA DE ENTREGA LÍMITE: 4/11/2026

CALIFICACIÓN: _____

EJERCICIOS DE SESIÓN:

1. Actividades realizadas. Una descripción de los ejercicios y capturas de pantalla de bloques de código generados y de ejecución del programa

Para el primer ejercicio retomé el holocrón que había creado anteriormente, modificando su representación para que pudiera realizar la animación del GIF proporcionado. En el código original el modelo se dibujaba como una sola malla, por lo que incorporé los modelos baseholocron y esquinageneral, cargándolos mediante la clase Model para representar la base y las ocho esquinas de manera independiente.

Posteriormente agregué las variables faseHolocron, aperturaHolocron y giroEsquinas, utilizando el tiempo obtenido con glfwGetTime() para controlar la animación. Dividí el movimiento en tres etapas: primero las esquinas se separan de la base mientras giran, después regresan a su posición original y finalmente el holocrón permanece en reposo durante un breve periodo antes de repetir el movimiento.

También utilicé una función de suavizado para controlar la separación y rotación de las piezas, evitando que los movimientos fueran bruscos. Para esto modifiqué las matrices de transformación de cada esquina mediante glm::translate y glm::rotate, conservando la posición de la base y permitiendo que las piezas regresen a su acomodo inicial.

```
// Animacion del Holocron: apertura, giro, cierre y pausa
GLfloat faseHolocron = (GLfloat)fmod(glfwGetTime(), 6.0);
GLfloat aperturaHolocron = 0.0f;
GLfloat giroEsquinas = 0.0f;

if (faseHolocron < 2.0f)
{
    // Abre y gira hacia afuera (0 a 2 segundos)
    GLfloat t = faseHolocron / 2.0f;
    GLfloat smooth = t * t * (3.0f - 2.0f * t);
    aperturaHolocron = smooth;
    giroEsquinas = smooth * 720.0f; // 720 grados = 2 vueltas completas
}
else if (faseHolocron < 4.0f)
{
    // Cierra y regresa el giro a la posicion exacta (2 a 4 segundos)
    GLfloat t = (faseHolocron - 2.0f) / 2.0f;
    GLfloat smooth = 1.0f - (t * t * (3.0f - 2.0f * t));
    aperturaHolocron = smooth;
    giroEsquinas = smooth * 720.0f; // Regresa las 2 vueltas suavemente
}
```

```
else
{
    // Pausa inactiva (4 a 6 segundos)
    aperturaHolocron = 0.0f;
    giroEsquinas = 0.0f;
}

GLfloat separacionHolocron = 1.0f + 0.35f * aperturaHolocron;
```

```
// HOLOCRON
//Modelo Inicial
color = glm::vec3(0.0f, 1.0f, 0.0f);
model = glm::mat4(1.0);
model = modelaux;
color = glm::vec3(0.0f, 1.0f, 1.0f);
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(20.0f, 8.0f, -16.0f));
modelaux2 = model;
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
baseholocron.RenderModel();
// Esquina 1 Rojo
model = modelaux2;
model = glm::translate(model, glm::vec3(5.1f * separacionHolocron, 5.2f * separacionHolocron, 5.1f * separacionHolocron));
color = glm::vec3(1.0f, 0.0f, 0.0f);
model = glm::rotate(model, -43.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, -54.0f * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(giroEsquinas), glm::vec3(0.0f, 1.0f, 0.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
esquinageneral.RenderModel();
// Esquina 2 Rosa
model = modelaux2;
model = glm::translate(model, glm::vec3(-5.1f * separacionHolocron, 5.2f * separacionHolocron, -5.1f * separacionHolocron));
color = glm::vec3(1.0f, 0.0f, 1.0f);
model = glm::rotate(model, 135.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, -54.0f * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(giroEsquinas), glm::vec3(0.0f, 1.0f, 0.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
esquinageneral.RenderModel();
```

```
// Esquina 3 Amarilla
model = modelaux2;
model = glm::translate(model, glm::vec3(-5.1f * separacionHolocron, -5.2f * separacionHolocron, 5.1f * separacionHolocron));
color = glm::vec3(1.0f, 1.0f, 0.0f);
model = glm::rotate(model, 43.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, -234.0f * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(giroEsquinas), glm::vec3(0.0f, 1.0f, 0.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
esquinageneral.RenderModel();
// Esquina 4 Naranja
model = modelaux2;
model = glm::translate(model, glm::vec3(-5.1f * separacionHolocron, -5.2f * separacionHolocron, -5.1f * separacionHolocron));
color = glm::vec3(1.0f, 0.5f, 0.0f);
model = glm::rotate(model, -45.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, -234.0f * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(giroEsquinas), glm::vec3(0.0f, 1.0f, 0.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
esquinageneral.RenderModel();
// Esquina 5 Verde
model = modelaux2;
model = glm::translate(model, glm::vec3(-5.1f * separacionHolocron, 5.2f * separacionHolocron, 5.1f * separacionHolocron));
color = glm::vec3(0.0f, 1.0f, 0.0f);
model = glm::rotate(model, -133.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, -54.0f * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(giroEsquinas), glm::vec3(0.0f, 1.0f, 0.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
esquinageneral.RenderModel();
```

```
// Esquina 7 Morado
model = modelaux2;
model = glm::translate(model, glm::vec3(5.2f * separacionHolocron, -5.1f * separacionHolocron, 5.2f * separacionHolocron));
color = glm::vec3(0.5f, 0.0f, 0.5f);
model = glm::rotate(model, -225.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, -235.0f * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(giroEsquinas), glm::vec3(0.0f, 1.0f, 0.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
esquinageneral.RenderModel();
// Esquina 8 Cafe
model = modelaux2;
model = glm::translate(model, glm::vec3(5.1f * separacionHolocron, -5.2f * separacionHolocron, -5.1f * separacionHolocron));
color = glm::vec3(0.8f, 0.5f, 0.2f);
model = glm::rotate(model, -138.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, -235.0f * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(giroEsquinas), glm::vec3(0.0f, 1.0f, 0.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
esquinageneral.RenderModel();
```

Para el segundo ejercicio incorporé un modelo de cohete espacial, cargándolo mediante la clase Model con el archivo Rocket.obj. Posteriormente agregué una animación controlada por la variable tiempoCohete, que se incrementa utilizando pasoAnimacion para calcular el movimiento del modelo durante la ejecución del programa.

Dividí el movimiento en tres etapas: inicialmente el cohete permanece sobre su plataforma de lanzamiento, después realiza el despegue y sigue una trayectoria parabólica, y finalmente desciende hasta llegar a una nueva posición donde permanece detenido. Para calcular su desplazamiento utilicé una variable llamada avanceCohete, que permite normalizar el tiempo de vuelo y obtener la altura mediante una ecuación parabólica. También agregué un desplazamiento sobre el eje Z para que el cohete avanzara durante su recorrido.

Posteriormente calculé la inclinación del cohete utilizando la pendiente de la trayectoria y la función atan2, permitiendo que la nariz del modelo cambie de orientación conforme asciende y desciende. Finalmente apliqué las transformaciones de traslación y rotación mediante una matriz llamada model, actualizando su posición y orientación para representar el despegue, el vuelo y el aterrizaje.

```
// Cohete: despegue, trayectoria parabolica y aterrizaje
static GLfloat tiempoCohete = 0.0f;
tiempoCohete += pasoAnimacion;
if (tiempoCohete >= 22.0f) tiempoCohete = 22.0f;

GLfloat avanceCohete = 0.0f;
GLfloat alturaCohete = 0.0f;
GLfloat desplazamientoCohete = 0.0f;
GLfloat inclinacionCohete = 0.0f;

if (tiempoCohete < 10.0f)
{
    alturaCohete = 0.0f;
    desplazamientoCohete = 0.0f;
    inclinacionCohete = 0.0f;
}
```

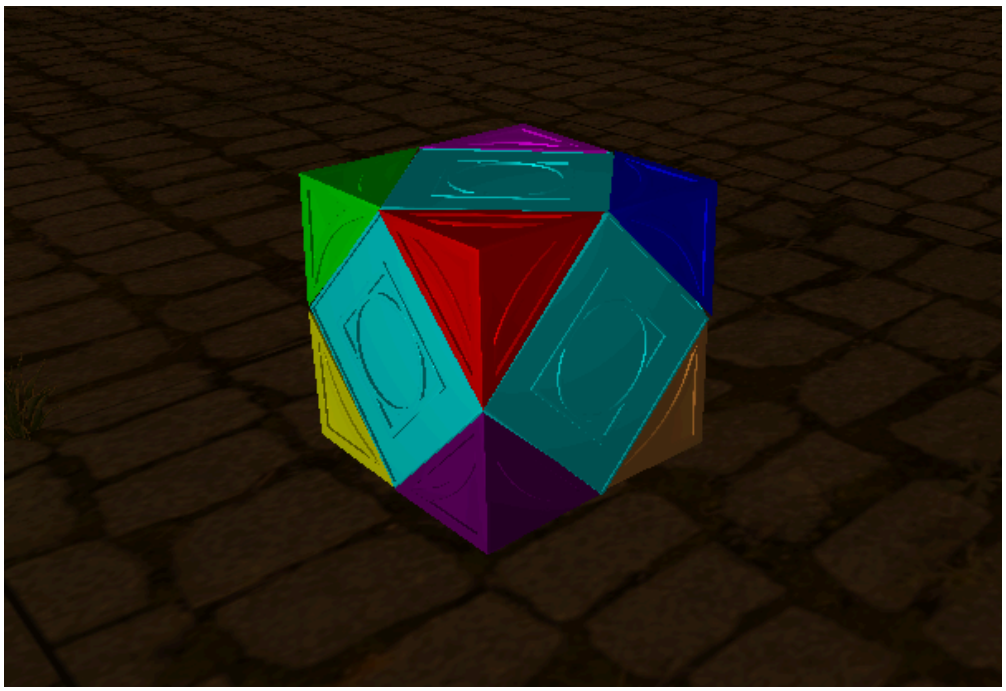
```
else if (tiempoCohete <= 20.0f)
{
    // Vuelo parabólico (segundo 10 al 20)
    avanceCohete = (tiempoCohete - 10.0f) / 10.0f;

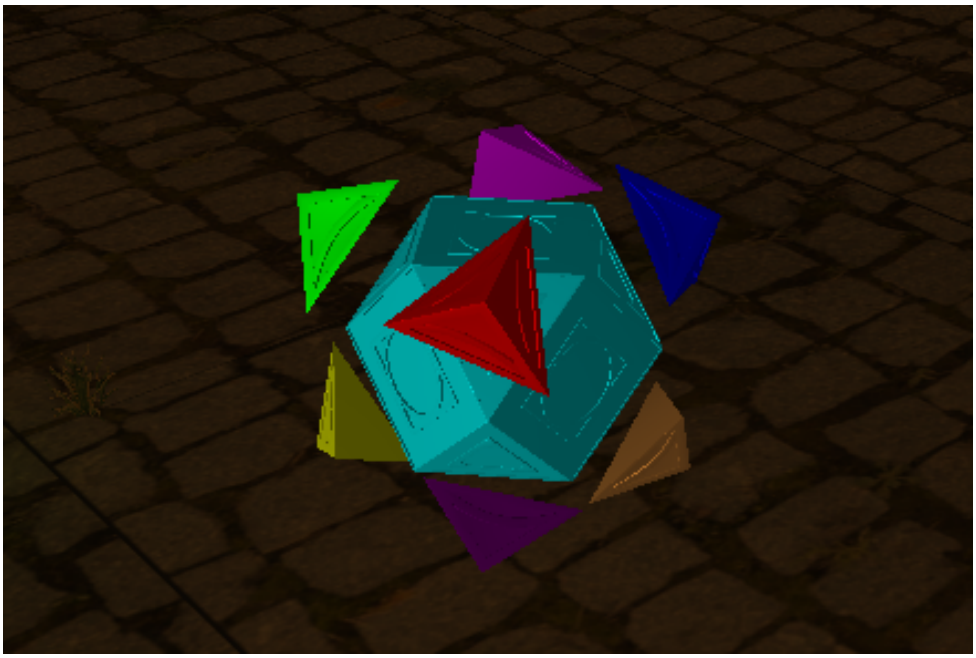
    // Trayectoria: Altura aumentada y desplazamiento lineal
    alturaCohete = 150.0f * 4.0f * avanceCohete * (1.0f - avanceCohete);
    desplazamientoCohete = -120.0f * avanceCohete;

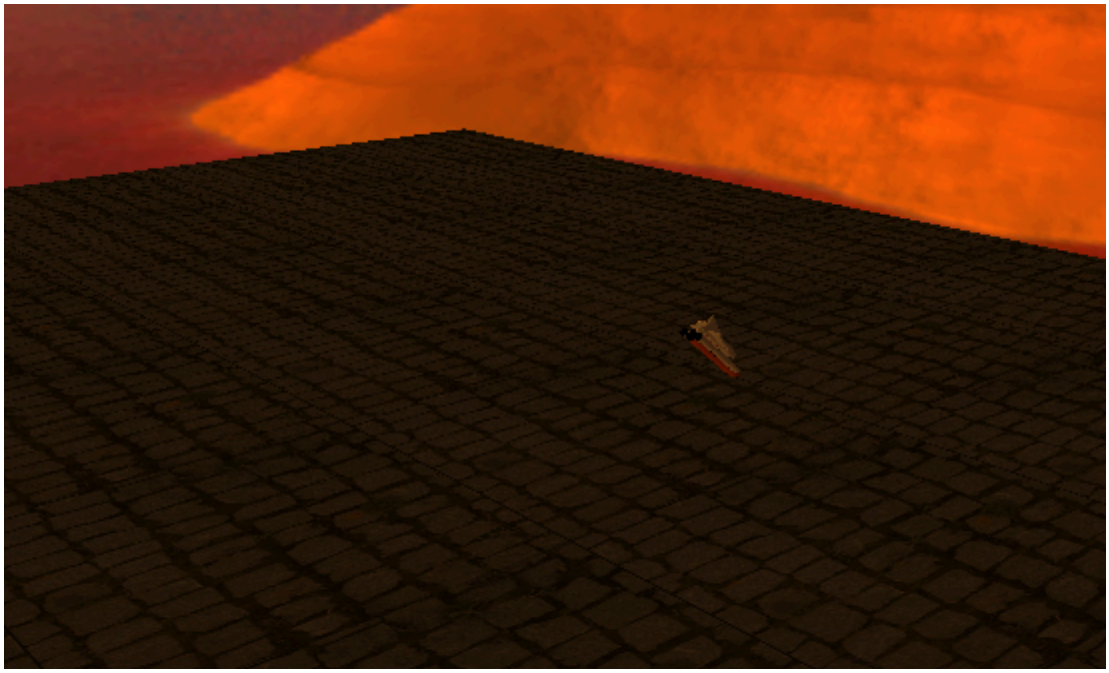
    // Inclinación dinámica basada en la derivada de la parábola
    GLfloat pendiente = 150.0f * 4.0f * (1.0f - 2.0f * avanceCohete);
    inclinacionCohete = atan2(pendiente, 120.0f);

    // Transiciones suaves de rotación para despegar y aterrizar derecho
    if (avanceCohete < 0.1f) {
        inclinacionCohete *= (avanceCohete / 0.1f);
    }
    else if (avanceCohete > 0.9f) {
        inclinacionCohete *= ((1.0f - avanceCohete) / 0.1f);
    }
}
else
{
    // Reposo estacionado en la zona de aterrizaje (segundo 20 en adelante)
    alturaCohete = 0.0f;
    desplazamientoCohete = -120.0f;
    inclinacionCohete = 0.0f;
}
```

La ejecucion se observa a continuacion:









2. Problemas presentados.

Durante el desarrollo de la práctica fue necesario ajustar las transformaciones de las esquinas del holocrón para conseguir que se separaran y regresaran correctamente a su posición original. También realicé ajustes en la trayectoria del cohete y en el cálculo de su inclinación para conseguir que el modelo despegara, siguiera el recorrido establecido y aterrizara de manera adecuada.

3. Conclusión:

La realización de esta práctica me permitió comprender mejor cómo utilizar el tiempo y las funciones matemáticas para generar animaciones automáticas en OpenGL. La animación del holocrón me ayudó a reforzar el manejo de modelos independientes y sus transformaciones, mientras que el movimiento del cohete me permitió aplicar ecuaciones para calcular trayectorias y orientar un modelo durante su recorrido. Con estos ejercicios pude ampliar mis conocimientos sobre las animaciones y el uso de matrices para controlar diferentes movimientos dentro de una escena tridimensional.